

• 解答

一、

1. 不够准确。样本为随机挑选的15名男生和15名女生的平均工资而非任意性别的30人的平均工资，而此性别比例与总体中的性别比例可能并不相同，且不同性别的人的平均工资也可能并不相同，由此可能引入由性别引起的平均工资的偏差；同时该样本量30只为总样本量300的10%，可知样本量较小可能有较大方差，因此此样本并不一定能准确反映总体的平均工资。

2. 我们假设该商品 t 日销量为 x_t ， t 日30日销量为 S_t ，10月1日记为 $t = 1$ 日。

根据30日销量定义，有

$$S_t = \sum_{i=t-29}^t x_i,$$

而相邻两日的30日销量之差为

$$S_t - S_{t-1} = x_t - x_{t-30},$$

推导得该商品 t 日销量为

$$x_t = S_t - S_{t-1} + x_{t-30}.$$

由于 $t \leq 30$ 日商品销量数据缺失，我们不妨做出假设估计前30日商品销量均与第31日的一致，即

$$x_{t-29} = x_{t-28} = \cdots = x_0 = x_1.$$

由此我们可以计算出迭代关系

$$x_1 = \frac{S_1}{30},$$

$$x_t = S_t - S_{t-1} + x_1, 2 \leq t \leq 30,$$

$$x_t = S_t - S_{t-1} + x_{t-30}, 31 \leq t \leq 92.$$

而每一日的销量数据可具体用已知的30日数据表示为

$$x_1 = \frac{S_1}{30}, t = 1,$$

$$x_t = S_t - S_{t-1} + \frac{S_1}{30}, 2 \leq t \leq 31,$$

$$x_t = S_t - S_{t-1} + S_{t-30} - S_{t-31} + \frac{S_1}{30}, 32 \leq t \leq 61,$$

$$x_t = S_t - S_{t-1} + S_{t-30} - S_{t-31} + S_{t-60} - S_{t-61} + \frac{S_1}{30}, 62 \leq t \leq 91,$$

$$x_t = S_t - S_{t-1} + S_{t-30} - S_{t-31} + S_{t-60} - S_{t-61} + S_{t-90} - S_{t-91} + \frac{S_1}{30}, t = 92,$$

3. 首先注意到因变量 $y \in \{0, 1, \dots, n\}$, 且模型直接对累积概率 $P(Y \geq i | X = x)$, $i \in \{0, 1, \dots, n\}$ 进行计算而非单个类别的值的概率, 故此扩展模型所适用的数据应有 n 个有序离散类别, 同时对于不同的 $i \in \{0, 1, \dots, n\}$, β 的取值并不改变, 故 X 的变化对各 i 的影响程度也应相同。

二、

1. 函数定义:

```
fibonacci 1 = 1
fibonacci 2 = 1
fibonacci n = fibonacci (n-2) + fibonacci (n-1)
```

2. python代码实现

#对初始长度为 n 的全0列表不断添加数字1到 n , 同时逐层枚举所有可能的添加绝对位置

```
def permutation(n, m):
    r=[[0 for i in range(n)]] #初始化一个长度为n的全0的二维列表
    for i in range(1,n+1): #填入数字从1到n遍历
        for j in range(len(r)): #对包含数字<=i的排列进行遍历
            t=r[j] #获取目标操作对象
            for k in range(n): #对目标操作对象中的元素进行遍历
                if t[k] == 0:
                    r.append(t[:k] + [i] + t[k+1:]) #替换0元素为数字i
            del r[j] #移除当前层的操作对象
    return tuple(r[m-1]) #返回第m个排列的元组形式
```

测试实例:

```
In [36]: # 对初始长度为n的全0列表不断添加数字1到n, 同时逐层枚举所有可能的添加绝对位置
def permutation(n, m):
    r=[[0 for i in range(n)]] #初始化一个长度为n的全0的二维列表
    for i in range(1,n+1): #填入数字从1到n遍历
        for j in range(len(r)): #对包含数字<=i的排列进行遍历
            t=r[j] #获取目标操作对象
            for k in range(n): #对目标操作对象中的元素进行遍历
                if t[k] == 0:
                    r.append(t[:k] + [i] + t[k+1:]) #替换0元素为数字i
            del r[j] #移除当前层的操作对象
    return tuple(r[m-1]) #返回第m个排列的元组形式
permutation(5, 3)
```

Out[36]: (1, 2, 4, 3, 5)